

3

Visualization in Python

Introduction to Graphs and Plots

CHAPTER OBJECTIVES
■ Gain an appreciation for the importance of visualization in data analysis and data science. ■ Appreciate how graphs can communicate essential information that tables could never. ■ Understand how perception of a data set or research finding can be even more important than the data itself. ■ Why when it comes to scientific tables and graphs, simple is usually better than complex. ■ Why percentage increases or percentage decreases can be misleading. ■ How to produce scatterplots, histograms, and other simple but effective tools of visualization. ■ A survey of bubble graphs, pie charts, heatmaps, correlograms, and other graphical tools.

In this chapter, we briefly explore some of the numerous visualization possibilities in Python and feature a discussion of graphics and data visualization in general. In reality, we only scratch the surface, as the capacities for visualization in Python greatly exceed what we can accomplish here. The importance of visualization in data analysis and data science cannot be overstated. **Graphs are essential to the effective communication of data.** As pointed out by many (e.g. see Tufte, 2011), however, graphs can just as well **mislead** as they can elucidate, and hence it is imperative that we develop an acuity and aptitude for being able to decipher exactly what a graphic is or is not communicating. Especially in these days of online visualizations literally bombarding us every day, the average consumer of a graph is likely to only spend a **few seconds** interpreting an image before redirecting his or her mouse to another application. Hence, the **perception** yielded by a graph can be long-lasting, even if that perception is faulty. For example, you may see a graph on the effects of COVID-19, and without realizing that you have misinterpreted it, carry that perception with you possibly forever! You never once critically evaluate or revise your perception. Then, when you are conversing with someone about COVID-19, you implement that

perception into a discussion as if it were reality! It is a faulty perception, yet you are carrying on with it as though it were **real**. As with the communication of any and all information, if the perception of what is being communicated is faulty, then regardless of the truth behind the information, it is the perception that will unfortunately dominate. In this sense, **perception becomes reality**. Therefore, how you choose to communicate information, whether scientific or otherwise, can have tremendous consequences. Statistics and visualization are especially good at misleading, if not used judiciously.

As an example of how powerful graphical perception can be, consider the following image comparing the magnitude of nuclear weapons in 1945 compared to 2020. How much energy and firepower existed in nuclear weapons at the time relative to today? Reporting a numerical measurement would likely not be that meaningful because it lacks context. On the other hand, a graphic representing a comparison of nuclear energy in the form of a basic visualization is much more enlightening (see Figure 3.1).

Regarding this plot, had the author simply cited a number in kilotons or similar units, for instance, unless the reader is a scientist familiar with the area, he/she does not leave the graph with a strong perception. **Visualizations have the power to convey a sense of urgency**. This visualization makes one go “Wow!,” something a number could never do. As we will see, Python is more than capable of generating quite effective and beautiful graphics. We survey some of these throughout the chapter, while highlighting which **packages** are most essential in generating quality scientific graphics and also where to go to explore graphics beyond what we cover in the chapter.

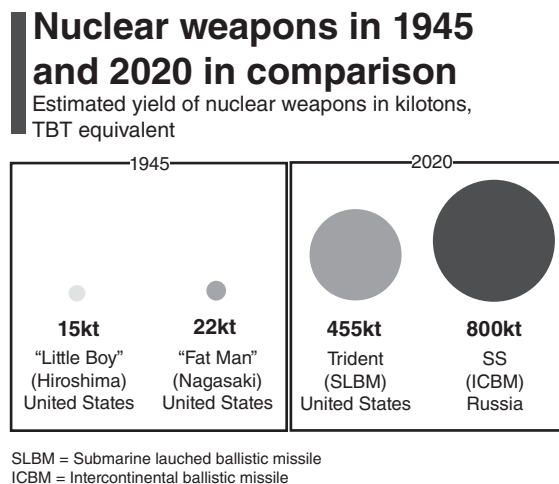


Figure 3.1 A Comparison of Nuclear Power in 1945 and 2020. Reproduced with permission from Statista (<https://www.statista.com/chart/3714/nuclear-weapons-in-comparison>).

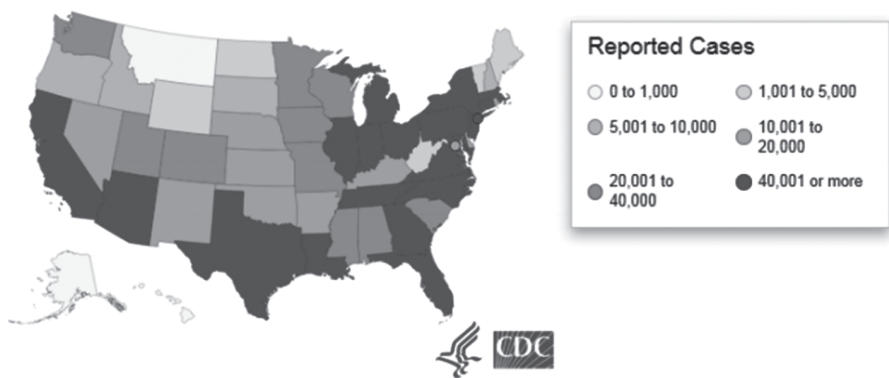
3.1 Aim for Simplicity and Clarity in Tables and Graphs: Complexity is for Fools!

As Dr. Malcolm in the film *Jurassic Park* once said, “Your scientists were so preoccupied with whether or not they **could**, they didn’t stop to think if they **should**!” The same is true for graphs. Just because software allows us to “dress up” graphics in the most interesting and complicated ways does not mean we should, and in most cases adding **aesthetics** to a graphic for the purpose of making it look **complex** is the sign of an amateur researcher rather than a professional scientist. Good graphs should be built with the clearest, ethical, and most effective honest communication motives possible in mind, not with the intention of making a fashion statement and manipulating the perceiver by way of complexity. Next time you see a graphic, and it is such a beautiful picture, ask yourself whether the author could have communicated the finding in a more efficient and parsimonious way. Too often, you will find the graph was constructed to “impress” rather than to communicate. This is truly unfortunate. Fortunately, good scientific journals will usually balk at such graphics and request instead graphs that prioritize a clear and ethical transfer of information from the author to the reader. What is perhaps most concerning about poorly constructed graphics is that often they may contain correct information, but it is simply that the information that they are communicating is unclear or purposely misleading. Again, it is all about what is perceived, not necessarily what the table or graph actually contains (that “perception becomes reality” idea we earlier alluded to).

For example, consider Table 3.1 describing “hot spots” for COVID-19 as of June 22, 2020. At first glance, it would appear that Montana especially is a true hot spot and cases must be surging due to the 316.7% increase. Though the percentage increase figure is correct, the perception one may be gleaned from the table, if not careful, can be completely misguided unless one simultaneously considers competing statistics. It turns out that the seven-day rolling average (yet another COVID-19 statistic) of daily cases in Montana was only equal to 17.9 for the period under review. Compare this with Texas, for instance, where the seven-day average was 3,939.9. However, Texas only had a 153.7% increase over 14 days, much less than that of Montana’s. So what is going on? What is going on is that getting a large **percentage increase** in Montana is not that difficult, since there are relatively few people who live in Montana to begin with (winter storms in late September (2019) and a high of freezing at Halloween in the same year apparently does not always encourage migration). However, remember that from 1 to 2 is a 100% increase. That is, 2 is 100% more than 1. Likewise, increasing from 1 to 3 is a 200% increase. The **absolute increase** in both cases is still exceedingly small. On the other hand, 1,000 to 1,500 is only a 50% increase, even though we are adding 500 more units. **Percentage increases or decreases can be extremely misleading, and hence must be interpreted with caution, preferably with sideline knowledge of the corresponding absolute increases.** While the percentage increase is informative, it does not tell the full story and can lead to a faulty perception of reality. A look at the total number of overall cases (Figure 3.2) reveals that Montana was one of the states with the fewest numbers, as shown (note that Montana is only lightly shaded, indicating total cases in the range of 0 to 1,000).

Table 3.1 Percentage increases in COVID-19 in 14 days as of June 22, 2020.

State	14-Day Percentage Change in Cases
Alabama	82.2%
Arizona	135.2%
California	52.2%
Florida	183.5%
Georgia	63.5%
Hawaii	233.3%
Idaho	144.9%
Kansas	77.7%
Mississippi	25.5%
Missouri	26.4%
Montana	316.7%
Nevada	98.7%
Ohio	38.8%
Oklahoma	266.6%
Oregon	103.7%
South Carolina	137.1%
Texas	153.7%
Utah	42.0%
West Virginia	72.9%

**Figure 3.2** Total reported COVID-19 cases. Source: CDC (Centers for Disease Control and Prevention). Used with Permission. Available at CDC (www.cdc.gov) free of charge.

Percentage increases in general (and not specifically related to COVID-19) are also often stated statistics for general **population growth** by state. Over the past few years (as of the time of this writing), much has been made about the population growth of southern Idaho. Apparently, Californians are moving to Idaho in droves. It has often been reported as the fastest growing state in the country, growing by approximately 2.5% over the past few years as of the time of this writing. However, a 2.5%

increase is still quite small in absolute numbers if the base from which the growth is taking place is quite small. “Idaho is growing like crazy!” media will report and yet the total population of the entire state as of 2020 is approximately one-fifth of the total population of New York City and not even one-half of the total population of Los Angeles! Yes, growing at a fast rate, but that is easy to do when the base is not very much to begin with.

Likewise, Bozeman, Montana, is apparently, again as of the time of this writing, one of the fastest growing towns in the USA. Total population as of 2020? Approximately 52,000! In 2010 the city had about 37,000 residents. Hence, in a 10-year period the city grew by approximately 40%, since 15,000 (i.e. 52,000–37,000) is approximately 40% of 37,000. A 15,000 increase in residents is really not that much at all over a 10-year period when you think about it. The **rate of growth** is large, but the absolute increase is still rather small. Therefore, the Internet headline of “Bozeman is fastest growing city and has grown by 40% in the last 10 years” is 100% accurate, but entirely misleading if you are taking something away (your perception) from the data that is not actually there. To a naïve consumer, the headline report of it being the fastest growing town may be enough to carry with them the perception that the town is being overrun by flocks of people. Hardly the case. Bozeman is growing, but the number of cows in the city limits (approximately 3 to 1 apparently) still far outweighs the number of residents! **Beware of percentage changes and interpret them with caution.** Or, at minimum, pair them up with absolute changes in the numbers to get a better feel for the data. Always be mindful of results you are interpreting in data analysis and be sure to put them into **context** of what is actually being communicated. Always ask for more context when interpreting any data or scientific result. It will usually reveal a new “base” (i.e. a new “denominator-type” number) upon which you can then more intelligently interpret the result before you. The number you are interpreting always has a denominator somewhere, even if not reported by the research.

3.2 State Population Change Data

To demonstrate a basic analysis using population change data, the following is census data on a number of indicators related to population increases or decreases by state in the United States. The data is available at **census.gov** (US Census Bureau, Population Division). We show only the first five cases of this data:

```
import pandas as pd
data = pd.read_csv("population.csv")
data.head()
Out[17]:
```

	SUMLEV	REGION	DIVISION	...	NRANK_PPCHG2017	NRANK_PPCHG2018	NRANK_PPCHG2019
0	10	0	0	...	X	X	X
1	20	1	0	...	4	4	4
2	20	2	0	...	3	3	3
3	20	3	0	...	1	1	1
4	20	4	0	...	2	2	2

```
[5 rows x 67 columns]
```

We will define and extract the **state** variable through the following, listing only a few of the cases at the top and then at the bottom of the file:

```
state = data['NAME']
state

Out[30]:
0          United States
1      Northeast Region
2      Midwest Region
55          Wyoming
56      Puerto Rico
Name: NAME, dtype: object
```

We will also extract the population change for 2019. This variable is named **PPOPCHG_2019** in the data file and we call it **pop_change_2019**, again listing only a few of the cases. Notice that Python allows us to simply name a variable by pulling it out of a dataframe. We could have named **pop_change_2019** something else had we wanted to, so long as we are still pulling **PPOPCHG_2019** out of the file:

```
pop_change_2019 = data['PPOPCHG_2019']
pop_change_2019

Out[32]:
0      0.475078
1     -0.113864
2      0.135376
3      0.811608
```

We can see that, overall, the United States has grown by 0.475% (i.e. the first number in the list), while the Northeast region has decreased by 0.113% (the second number), and so on. While the above is the percentage increase or decrease, below are the actual **raw numbers** corresponding to this increase or decrease. The raw numbers are given by the variable **NPOPCHG_2019**:

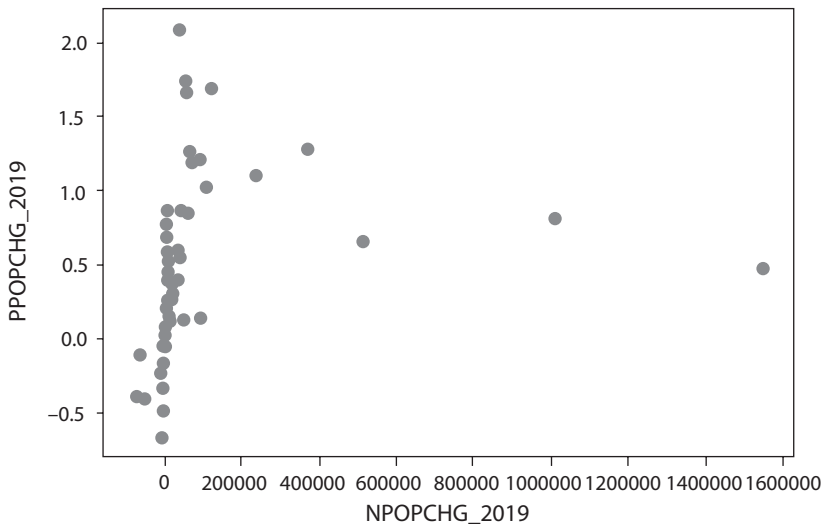
```
N_pop_change_2019 = data['NPOPCHG_2019']
N_pop_change_2019

Out[47]:
0      1552022
1       -63817
2        92376
3      1011015
```

That is, the total increase in population in the United States is 1,552,022, for the Northeast region a decrease of 63,817, and so on. Notice that the percentage increase or decrease, other than the positive or negative signs, does not tell us the full story. The raw numbers give us the true sense of the **magnitude of change**. Indeed, if we produce a scatterplot of the two variables to visualize how percentage change is associated with actual numerical change, we obtain the following:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
plt.plot(data["NPOPCHG_2019"], data["PPOPCHG_2019"], "o")
plt.xlabel("NPOPCHG_2019")
plt.ylabel("PPOPCHG_2019")
Out[26]: Text(0, 0.5, 'PPOPCHG_2019')
```



We see that a few points associated with large raw numbers are not necessarily the ones associated with huge percentage increases or decreases, otherwise the plot would reveal that as the raw numbers increase, the percentage change would also increase. As we can see, this is not the case. A percentage increase or decrease is not necessarily related to an absolute increase or decrease, again highlighting the point just discussed. Hence, the news report that gun sales have increased by 200% in a small town, for instance, may simply mean a few more people bought guns, but still a very inconsequential increase in absolute number. On the other hand, an increase of 20% in gun sales in New York City would imply a lot more guns on the streets, simply because the base from which we are expressing the percentage increase is much larger. For this increase, the absolute number will also be greater in number.

3.3 What Do the Numbers Tell Us? Clues to Substantive Theory

Returning again to the COVID-19 data, recall that the total number of cases may be equally or even more informative than the percentage change, as the following graph depicts for the period ending October 9, 2020. The shading represents the number of cases of COVID-19 per 100,000 people by county in the state of California, where a darker shade represents more cases than a lighter shade (Figure 3.3).

At first glance, it may be argued that what is most vital in COVID-19 case information is the number of cases per square mile or some other measure of **density** information. Although this information might be useful, it is less likely to be of value for people



Figure 3.3 Map of COVID-19 Outbreak in the State of California as of October 9, 2020, where a darker shade represents more cases than a lighter shade. (Source: California Department of Public Health. Used with Permission.)

who want to know where they may have a greater chance of contracting the illness, since it is highly contagious. **Reasons** or **causes** as to why some counties have higher numbers and potentially higher densities than other counties is, of course, a complex question. At first glance, it may appear that potentially **higher temperatures** in the southern part of the state (assuming those temperatures are higher) are not subduing the virus, as was originally thought they might. Recall it was originally hypothesized early in 2020 that with increasing temperature, the virus may not survive or spread and would be more limited than in the winter months when the virus presumably began. However, the density information may encourage us to ask further questions of the data, such as the following:

- Are counties in the south more populated than counties in the north? Hence if temperature is diminishing spread, perhaps it does not have as much opportunity to do so in the north as much as in the south.

- What were the social distancing and mask requirement regulations in the different counties? Did the southern counties have more stringent or less stringent requirements than the north?

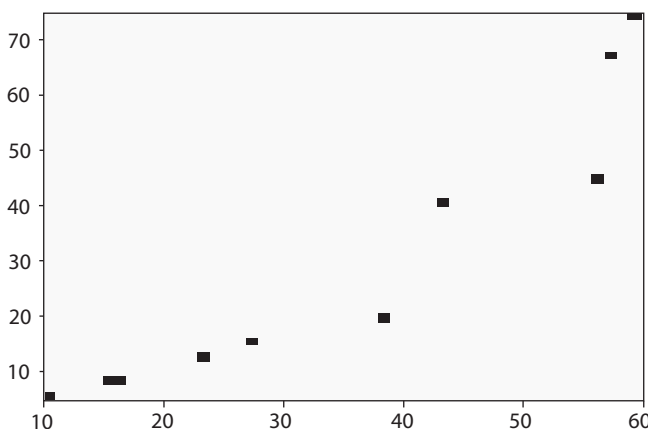
We can see through examples such as this that graphics rarely answer questions, but may help us understand further **which new questions to ask**. That is, graphics and visualizations may help us conjure up new **hypotheses** that we can subject to data analysis to seek out answers. **Data itself, however, never equals theory**. In reference to our example, all we see is a greater number of individuals with COVID-19 in the southern parts of the state. The “how and why” behind those numbers is where theory comes in. Hence, you can perform data analyses all you like, but informing theory and obtaining scientific explanation can still be very challenging, especially if you are unable to perform a rigorous experiment to rule out many competing alternative hypotheses.

3.4 The Scatterplot

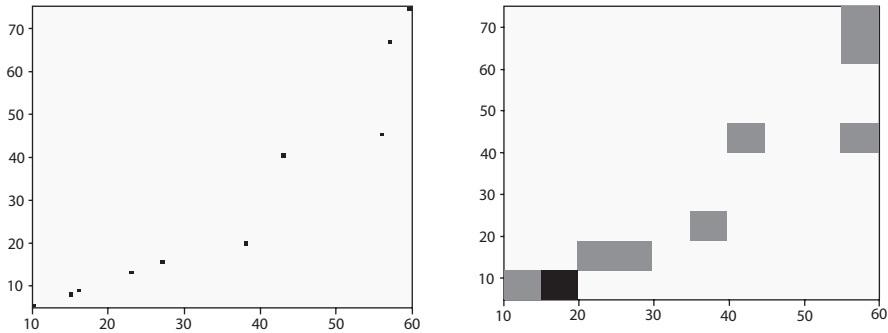
One of the most common and useful graphs in the history of statistics is that of the **scatterplot**. The scatterplot depicts the **bivariate relationship** between two variables or the **multivariate** relationship among many more variables (e.g. three-dimensional scatterplots), and hence it is appropriate for bivariate or multivariate data. However, it becomes unwieldy for visualizations in higher than three dimensions. The classic two-dimensional scatterplot can be easily generated using **matplotlib** in Python. **Matplotlib** is one of the more popular packages in Python for generating a variety of graphics, which include static, animated, and interactive visualizations. You can read more on **matplotlib** by referring to its website at **matplotlib.org**.

As an example of a simple scatterplot in Python, we build a plot from scratch on two generic variables x and y . The data for this example are fictitious:

```
x = [10, 15, 16, 23, 27, 38, 43, 56, 57, 60]
y = [5, 8, 9, 13, 16, 20, 40, 45, 67, 75]
import matplotlib.pyplot as plt
import numpy as np
plt.hist2d(x, y, bins=(50, 50), cmap=plt.cm.Reds)
Out[6]:
```



We can easily add a title to the plot quite simply by applying `plt.title("Scatterplot")`. We could also quite easily make adjustments to the plot, such as changing the number of bins. Let us change the bins from 50,50 to 100,100: `plt.hist2d(x, y, bins=(100, 100), cmap=plt.cm.Reds)` (below, left). Had we decreased the bins, we would get the following (below, right): `plt.hist2d(x, y, bins=(10, 10), cmap=plt.cm.Reds)`:



To add labels to the x and y axes, simply code `plt.xlabel("x")` and `plt.ylabel("y")` as shown earlier, where “x” and “y” here are just generic labels for convenience. If you were plotting weight by age, then you would rename them `plt.ylabel("weight")` and `plt.xlabel("age")`, and so on. As mentioned at the outset of the book, it is rarely worth your time memorizing code, at least for the purposes of using it for statistical and scientific analysis. What is worth your time, however, is learning how to operate with code, work with it, adjust it, explore it, and see what you can create. Then, as you work more with Python and perform new analyses, you become more experienced, and coding ideas and frameworks become more “available” to you, so at least you have ideas on where to begin. Even professional coders will not have all of this information at their fingertips, which is why their personal libraries are filled with coding books. Daily, they are thumbing through manuals (or the Internet) working with code.

Scatterplots, of course, do not by themselves provide a measure of the **degree of relationship** among variables, they simply reveal the scatter of the data. That is, they give us an idea of the **bivariate variability**, and nothing more. We delay the discussion of measuring bivariate relationships numerically (rather than graphically) to the following chapter where we discuss the Pearson correlation in some detail. The Pearson correlation is worthy of some attention because it is a commonly used measure for assessing the degree of relationship among variables, specifically linear ones. There, we also consider a non-parametric counterpart to Pearson, that of the Spearman rank correlation, to help evaluate relationships that may not be linear in form. We will unpack exactly what makes a correlation “tick” and what can vs. cannot be concluded from a correlation coefficient. As we discuss then, many times too much is made of a scientific result simply because a correlation is observed.

3.5 Correlograms

A **correlogram** is useful for depicting several correlations in the same visual field. As an example, consider the following plot on correlations among features of the **iris** data. For this plot, we use **matplotlib** and produce two versions of the plot. Both

functions utilize `sns.pairplot()`. We first import the libraries for generating the plots, specifically `matplotlib.pyplot` and **seaborn**:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Next, we request Python to load the iris data. The iris data here is contained in **seaborn**, so we are using the `load_dataset` function as the extension to **sns** (“sns” because this is what we abbreviated **seaborn** to above). We name the dataframe `df`:

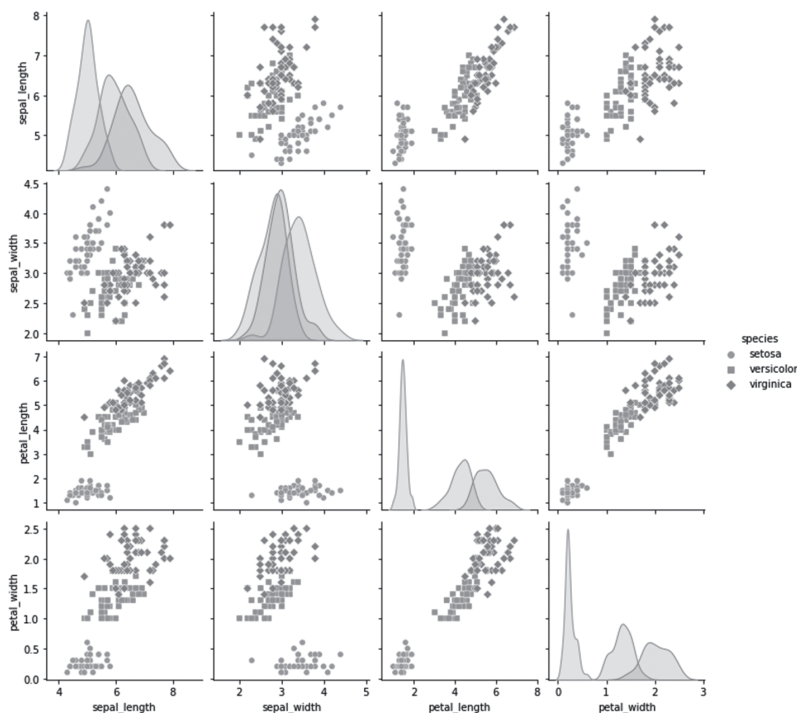
```
df = sns.load_dataset('iris')

df.head()
Out[150]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

The `df.head()` function requests the first few cases of the dataframe (e.g. `df.head(20)` would give us the first 20 cases, and so on). Let us get on with generating the correlogram. For this, we will use the `pairplot` function in **seaborn**:

```
sns.pairplot(df, kind="scatter", hue="species", markers=["o", "s",
"D"], palette="Set2")
plt.show()
```



As an exercise (plot not shown), try adjusting the plot further by the following to see what effect these changes have on the plot:

```
sns.pairplot(df, kind="scatter", hue="species", plot_kws=dict(s=80,
edgecolor="white", linewidth=2.5))
plt.show()
```

We can see that included in the correlograms are also univariate distributions along the main diagonal. The options for adjusting graphics such as these are almost endless, far beyond our capacity to detail here. For more details on seaborn pairplot, visit seaborn.pydata.org. We will use this plot later in the book when we seek to identify **clusters** in the iris data in our survey of **cluster analysis**.

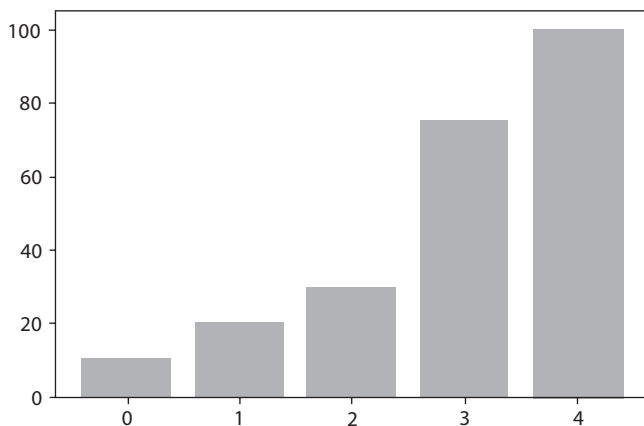
3.6 Histograms and Bar Graphs

Histograms are useful for plotting univariate data. Typically, a **histogram** implies that what is being measured on the x-axis is continuous, so that the histogram bars are allowed to touch one another. The touching implies the continuity. In contrast, **bar graphs** typically have a small space between bins, to imply that the data are not continuous, but instead **qualitative** or **categorical**. The space indicates that there can be nothing in between the categories. For example, there cannot be anything in between heads or tails on a flip of a coin. You cannot get a result in between (unless it lands on its edge I suppose, but we rule out that possibility). Likewise, if you are Democrat vs. Republican, the bar graph plotting the frequencies of these two categories equally implies a lack of continuity. We know you can be a moderate (i.e. the midpoint between Democrat and Republican), but the point is that a bar graph with these two categories implies that from Democrat to Republican there is no value in between these two categories, unlike the fact that the numbers 0 to 1 on the infinitely dense real line implies a continuum of values. The graph itself can indicate what values are possible along the scale for the given experiment, study, or data description. In practice, the distinction between what is being portrayed in the graphic in terms of continuity vs. discreteness may not always be clear from the given visualization and you may need to familiarize yourself with the data a bit more to learn whether the variable is continuous or not. Histograms and bar graphs are often used interchangeably and the definition we have given may not always be followed to such precision. Formality gives way to practicality in portraying information, so be sure to ask questions regarding what you are looking at when making the continuity vs. discreteness distinction.

In the following plot, we generate a bar graph and specify the heights of the bars as 10 through 100:

```
import numpy as np
import matplotlib.pyplot as plt
height = [10, 20, 30, 75, 100]
bars = ('A', 'B', 'C', 'D', 'E')
y_pos = np.arange(len(bars))
plt.bar(y_pos, height, color=(0.2, 0.4, 0.4, 0.6))

Out[64]: <BarContainer object of 5 artists>
```



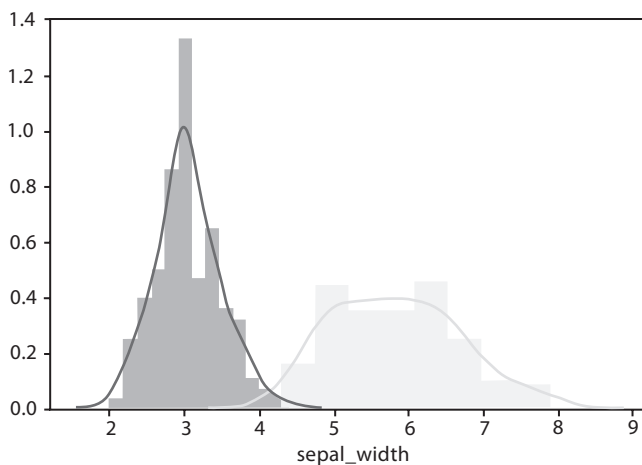
One can easily modify the entries of the plot by adjusting any of the several parameters in it.

3.7 Plotting Side-by-Side Histograms

Sometimes we would like to see histograms next to each other representing distributions of different groups. We can use **seaborn** for this, specifically `distplot`. We use the data **iris** to demonstrate:

```
import seaborn as sns
df = sns.load_dataset('iris')

sns.distplot(df["sepal_length"], color="skyblue", label="Sepal Length")
sns.distplot(df["sepal_width"], color="red", label="Sepal Width")
plt.show()
```

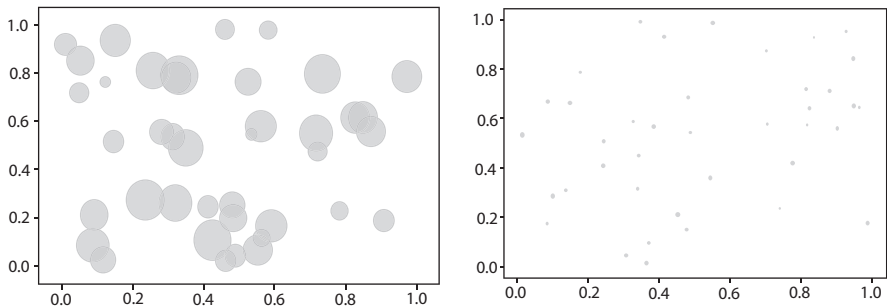


3.8 Bubble Plots

The **bubble plot** allows us to visualize a third variable in the plot while plotting the relationship between two other variables. In the case we consider here, it is read as an ordinary scatterplot, but the size of the bivariate points (i.e. the “bubbles”) reveals information about a third variable in terms of **magnitude**. Here we plot a simple plot using random numbers:

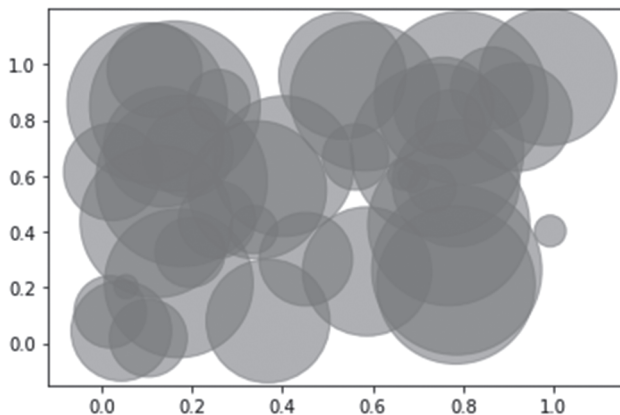
```
x = np.random.rand(40)
y = np.random.rand(40)
z = np.random.rand(40)

plt.scatter(x, y, s=z*1000, alpha=0.5)
plt.show()
```



We can easily change the relative sizes of the bubbles by adjusting $s=z*1000$. For example, instead of equal to 1,000, if we size it down to 10, so that we have $s=z*10$, we obtain the plot above on the right. Note that if we make the marker s too large, the bubbles become too big and the plot is essentially useless:

```
plt.scatter(x, y, s=z*10000, alpha=0.5)
plt.show()
```



This emphasizes a common theme when generating graphics in software, such that you may have to “tweak” things a few times before you get it “just right” and the graphic communicates what you wish to say in an efficient manner. In the case of the

bubble plot, the best size of the bubbles may need some experimenting with before you land on the best option. Novices to data analysis often do not realize the time and energy it can take simply to create even **one** appealing graph when generating results.

Several varieties of bubble plots were especially useful and popular during the COVID-19 pandemic. Figure 3.4 is a plot produced early in the pandemic by Johns Hopkins University showing the number of cases per 10,000 population.

Again, graphing data can help with conjuring up new hypotheses as to the reasons why the concentrations were so high in certain regions but not others. For instance, in the United States, though we know the east coast's population density is greater than in the western states, it is also true that it is quite a bit drier in states such as Montana and Wyoming and the Dakotas. Could this explain the prevalence of COVID-19? Probably not, but the point is that what might appear to be one explanation for the data could quickly change if you consider a different variable or possibility. **One new variable can change everything when it comes to interpreting data or narrowing in on a scientific theory.** Determining true causes or reasons is very difficult, which is why data analysis on COVID-19's spread could not reveal causes. It could only conjure up further hypotheses. As we will see later in the book when we survey ANOVA and regression, even adding a single variable to a statistical model can change the effects of other variables. Hence, when you are seeking an explanation for data or events, you must always try for the most thorough picture you can get. You could have 99 variables at your disposal, yet it is the 100th one that you do not have that is the key to solving the scientific puzzle. Thus, when you model the first 99, whatever model ensues will necessarily be wrong, and if the 100th variable is of vital importance, the model based on the first 99 variables may be entirely misleading without it. We will discuss this issue and more as we progress throughout the book. **Model context matters a great deal.** It is essential that you learn what makes a statistical model “tick” and the extent to which it might (or might not) reflect scientific realities.

Coronavirus hot spots

Coronavirus cases, per capita, in counties with five or more confirmed cases. (Per 10,000 population, as of Apr. 14, 2020)

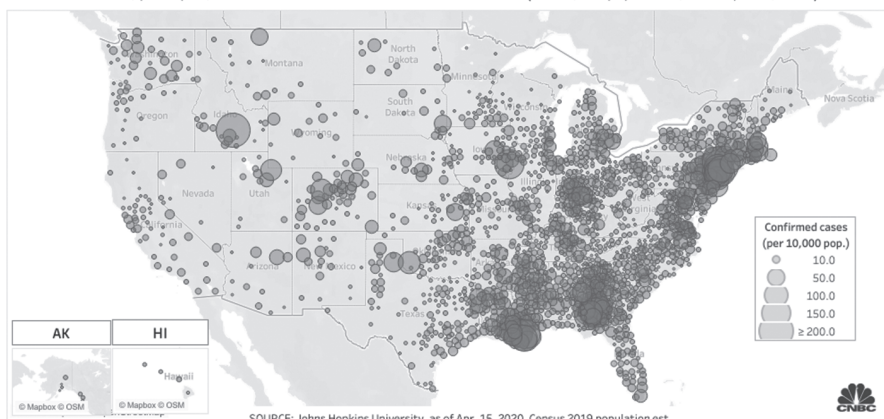


Figure 3.4 Bubble plot of COVID-19 cases across the United States early in the pandemic.
Source: Johns Hopkins CSSE. Used with Permission.

One weakness of bubble plots is that it may be difficult to decipher the sizes of the bubbles in relation to other bubbles, especially given overlapping bubbles. For instance, in the aforementioned plot, it becomes difficult to know from the plot which areas definitively had the most cases due to the similar-sized bubbles in the plot. Displaying a bar graph would reveal more regarding the **horizon** of the different categories than bubbles do. That is, instead of the bubble plot, one might instead generate a bar graph so that one could more easily see the degradations in density as we move from state to state. That “idea” is virtually impossible to “see” in the bubble plot.

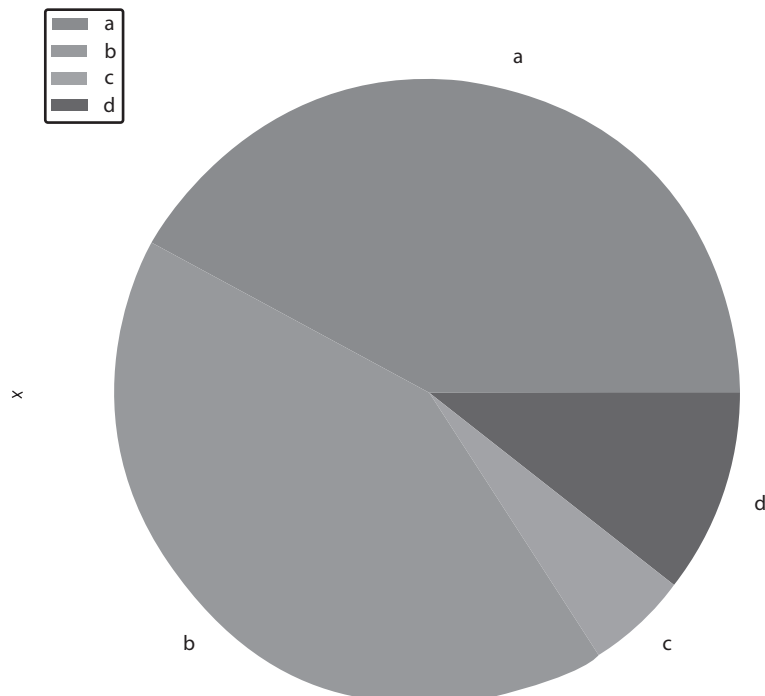
3.9 Pie Plots

Pie plots or pie “charts” are another common and popular graphing device (and a website favorite!), despite the fact that they can be quite problematic when it comes to the clear and effective communication of data. To demonstrate the problems with pie plots, consider the following plot, which we produced using **pandas** with categories **a** through **d**:

```
import pandas as pd
df = pd.DataFrame([8,8,1,2], index=['a', 'b', 'c', 'd'],
columns=['x'])
```

The function we use is `df.plot()`, specifying `pie`:

```
df.plot(kind='pie', subplots=True, figsize=(8, 8))
Out[64]:
array([<matplotlib.axes._subplots.AxesSubplot object at
0x0000000001A783550>],
      dtype=object)
```



How is the innocent pie graph problematic? In color (you will have to produce it yourself to see the colors), it certainly looks appealing, and we see it in magazines and online data descriptions all the time. However, the plot we just produced clearly reveals the difficulty. Similar to the bubble plot, it becomes difficult to decipher the size of the various “slices” unless they are quite distinct. To convince yourself of this, take a quick two-second glance at the plot and answer the question of which category, a or b, has the most frequency? It is almost impossible to tell this information from a quick glance at the plot, and when the numbers are not included in the slices, the plot is quite useless for making such distinctions. Much more useful would be a bar graph or histogram, where again the **horizon** is more apparent. Then we could immediately see which category has more frequency. Having said that, pie graphs and charts are an all-time favorite and they will likely be a mainstay of visualization techniques (especially outside of professional papers) for years to come. Simply be aware of their drawbacks and limitations when interpreting them.

3.10 Heatmaps

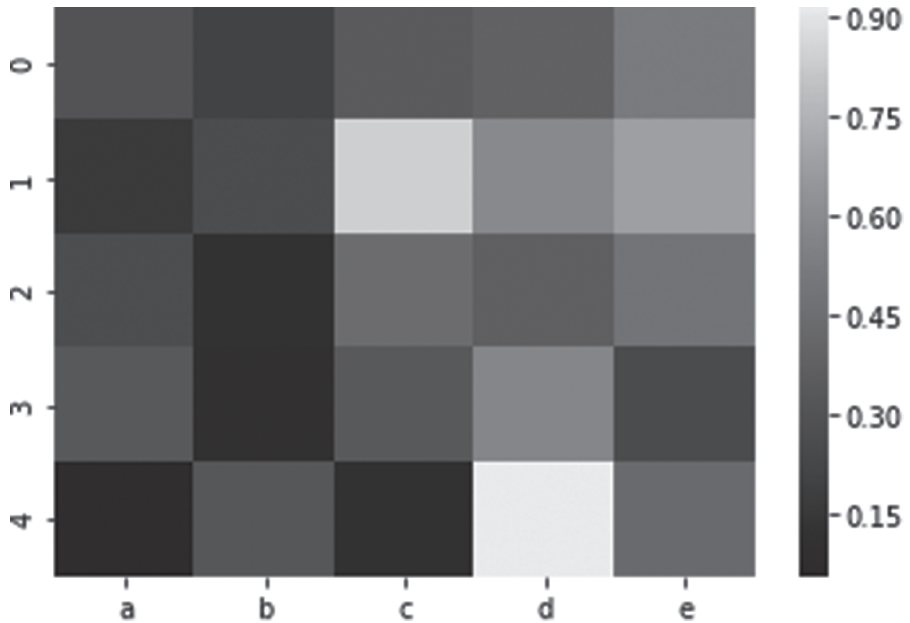
A **heatmap** is a graphic where a measure of magnitude or density is typically indicated in the plot. As an example of a heatmap in Python, let us create one on a random data set. First, we create the data with columns a through e:

```
import seaborn as sns
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.random((5,5)),
columns=["a","b","c","d","e"])
df
Out[161]:
a b c d e
0 0.304220 0.207444 0.348501 0.381417 0.524326
1 0.163985 0.263075 0.840032 0.592651 0.682820
2 0.265063 0.111662 0.445554 0.370213 0.483865
3 0.337630 0.100143 0.339747 0.579408 0.261271
4 0.056892 0.322099 0.111199 0.915698 0.429183
```

The above is now a matrix of five rows (0 through 4) by five columns (a through e). We now create a heatmap of this data using `sns.heatmap()`:

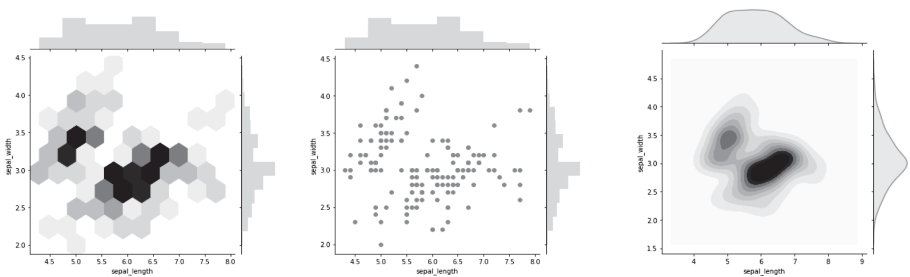
```
map = sns.heatmap(df)
```

It is easy to match up the heatmap with the above data layout. The smaller the number, the darker the shade. For example, the cell 4-a has a very dark shade. It is associated with a number of only 0.056892. On the other hand, the cell 4-d has a very light shade and is indicated with a much greater number, that of 0.915698. There is no limit to how the shading idea of a heatmap can be created or adjusted to reflect data. Similar plots can be very useful to reveal density of points in a scatterplot as well, where instead of jittering points on top of one another, density is revealed by shade. We demonstrate this using the **iris** data:



```
import seaborn as sns
df = sns.load_dataset('iris')

sns.jointplot(x=df["sepal_length"], y=df["sepal_width"], kind='hex')
sns.jointplot(x=df["sepal_length"], y=df["sepal_width"], kind='scatter')
sns.jointplot(x=df["sepal_length"], y=df["sepal_width"], kind='kde')
Out[3]: <seaborn.axisgrid.JointGrid at 0x1a284f98>
```



Notice that to generate the different plots, all we adjusted was the `kind = ''` category. For instance, the third plot featured `kde`, which stands for **kernel density estimate**. Which plot of the three is most useful? While they each communicate slightly different information, the leftmost plot (`kind = 'hex'` for “hexagonal bins”) is probably our favorite, as it combines a measure of relationship with density per cell. In all plots marginal distributions are plotted as well, represented by the bar graphs at the top and right side of the first two plots, and corresponding density plots in the third plot.

3.11 Line Charts

Line charts or line graphs are extremely common, and are especially excellent for revealing the course of a variable over **time**. They can be very useful for plotting repeated-measures data to track behavior over time, such as pre-test, post-test, and later timewise evaluations. Consider the line graph of the price of oil since 2019, featuring the historic crash of 2020 where the price of a barrel of oil actually turned negative (Figure 3.5).

What makes line graphs especially appealing is that they have a reasonable **horizon** (unlike those bubble plots and pie charts), meaning that it is (in most cases) easier to see which time points have the highest or least values. Cutting up the x-axis into increasingly more narrow slices is also always a possibility, though it may not always be a good idea because then the categories can get too thinly parsed (for example, suppose the x-axis was by day rather than by month in the oil plot). As always with graphs and plots, it is a trade-off, in that if we try to parse things out too “thinly,” we actually lose information, because we are too focused on the case-by-case variation (such as with a table of data with narrow interval sizes). On the other hand, a too “global” portrayal of the data does not allow us to see case-by-case variation nearly as well. **Always aim for a balance of sorts between these extremes to produce the best graph suited to your needs.** No graph will be perfect. It may require a lot of “tweaking” and adjusting before you settle on the graphic that most effectively meets your goals of what you wish to communicate with your data.

We can easily create a line graph in Python using `matplotlib.pyplot`:

```
import matplotlib.pyplot as plt
import numpy as np
```



Figure 3.5 The price of oil from 2019 to 2020. The price turned negative during the COVID-19 pandemic. *Used with Permission.*

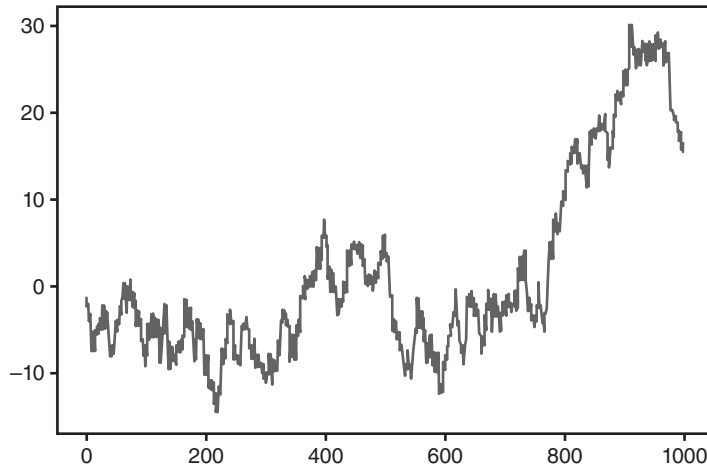
Let us first create some random numbers from the normal distribution using `np.random.randn()` with a range of 1 to 1,000 on the x-axis:

```
values=np.cumsum(np.random.randn(1000,1))
```

We now plot the resulting data, in which we can clearly see the rise and fall of the data across values on the x-axis:

```
plt.plot(values)
```

```
Out[4]: [<matplotlib.lines.Line2D at 0x1a8d89e8>]
```



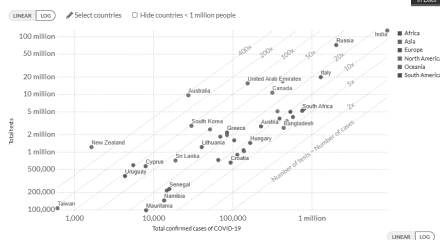
3.12 Closing Thoughts

In this chapter we have merely touched on a few of the possibilities provided by Python when it comes to graphics and visualization. Since the goal of this book is an introduction to applied statistics and data analysis rather than visualization in particular, space has allowed us to simply provide a glimpse into this vast world. Indeed, one can easily spend one's career developing new graphics and ways of visualization (many professionals choose to do exactly this), and combined with the increases in computer software and technology, the sky is the limit. If graphics and visualization interest you, you are strongly encouraged to pursue this exciting field. VanderPlas (2017) provides further detail and specifics on generating powerful visualizations using **Matplotlib** and **Seaborn** in particular, and should be consulted for more information. As always, if you need to produce a fancy graph for your research or publication, it is well worth delving into Python's graphing capabilities to brainstorm the best plot and explore many possibilities, not just one or two. You do not need to be a graphing expert to produce a great graph that is much better than "canned graphs" you can more easily obtain in Excel or SPSS. Open up your Python books, get online, and do some searching; that is what research and computing is all about!

Review Exercises

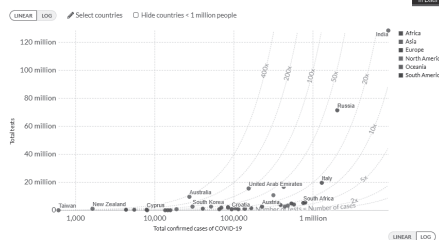
1. Discuss and explore the impact a **visualization** can have on one's **perception** of a data set. Whether or not the information gleaned from a graphic is correct or not, what are the benefits or dangers of it influencing one's actions and decisions?
2. Why is aiming for **simplicity** in graphics a good strategy? Why is unnecessary complexity not encouraged? (For example, why is a 3-D histogram graphic in MS Excel potentially not a good idea?)
3. Discuss why the idea of a **percentage change** can be very misleading when interpreting data. Refer to the COVID-19 example featured in the chapter and then see if you can come up with an example of two small data sets (of approximately 10 numbers each) to demonstrate the problem with interpretation. Why are raw numbers sometimes preferable over percentage changes?
4. For the population change census data explored in the chapter, compare the **percentage change** to the **absolute change** for Idaho vs. other states. After doing so, comment on why the statement "Idaho is one of the fastest growing states," though technically correct, can nonetheless be somewhat misleading.
5. Elementary as it may seem at first glance, understanding what a **scatterplot** does vs. does not tell you can be challenging. Consider the following two plots relating total confirmed cases of COVID-19 to total COVID-19 tests conducted, presented on both a **linear scale** and then a **log scale**. What are these data telling us, and why does the plot look different across the linear vs. log scales? That is, what is the impact of changing the **scaling** along the y-axis?

Total COVID-19 tests conducted vs. Confirmed cases, Nov 20, 2020



Source: Tests: official data collected by Our World in Data. Confirmed cases: European CDC - Shapton Update Worldwide. Our World in Data.org/ourworldindata.org CC BY. Note: Comparisons of testing data across countries are affected by differences in the way the data are reported. Details can be found at our Testing Dataset page.

Total COVID-19 tests conducted vs. Confirmed cases, Nov 20, 2020



Source: Tests: official data collected by Our World in Data. Confirmed cases: European CDC - Shapton Update Worldwide. Our World in Data.org/ourworldindata.org CC BY. Note: Comparisons of testing data across countries are affected by differences in the way the data are reported. Details can be found at our Testing Dataset page.

Published online at OurWorldinData.org. Retrieved from: <https://ourworldindata.org/grapher/covid-19-total-confirmed-cases-vs-total-tests-conducted>.

6. Using the **iris** data, generate a scatterplot in Python with sepal length on the x-axis and petal length on the y-axis. Does the relationship look linear? Estimate the strength of the linear relationship in the plot.
7. Plot a **histogram** of sepal length in the **iris** data and comment on the shape. Is the distribution approximately normal? Skewed? Now, plot the same data but this time with a **boxplot** (use `seaborn.boxplot()`). Compare the histogram to the boxplot. Which is more informative to you, and why?

8. Why are **bubble plots** potentially problematic? For example, for the data featuring the number of COVID-19 cases by country, how might a bar graph be more useful than a bubble plot here? Are there any disadvantages to plotting the bar graph instead of the bubble plot?
9. Derive a very small data set of 5–10 numbers that effectively illustrates the primary problem with **pie graphs**. Why would a “boring” histogram or bar graph be a better choice for your data?
10. When might producing a **heatmap** be useful over simply interpreting numbers in a table? In other words, why not simply look at the numbers in a 2×2 table (for instance) instead of plotting a heatmap?
11. Consider the statement, “Graphs, like statistical models, will never reveal the complete picture, and may reveal just enough of it to draw faulty conclusions.” Interpret and discuss this statement and invent a situation where **not graphing data** is probably a wise choice.